Clustering and Decision Trees

Introduction to Machine Learning

Module 4

Prof. Rajesh Nag

Compiled by Amitendu and Claude

Contents

1 Introduction to Clustering **2**
1.1 Clustering vs. Classification . . . 2
1.2 Applications . . . 2
1.3 Similarity Through Distance . . . 2
1.4 Main Families of Clustering Methods . . . 3
1.5 What Makes a Good Clustering? . . . 3

2 K-Means Clustering and the Centroid Concept **4**
2.1 The Centroid . . . 4
2.2 The K-Means Algorithm . . . 4
2.3 What K-Means Optimises . . . 5
2.4 Worked Example . . . 5
2.5 Choosing the Initial Centroids . . . 6
2.6 Choosing k . . . 6
2.7 Limitations of K-Means . . . 7
2.8 Code . . . 7

3 Decision Tree Structure **8**
3.1 Anatomy of a Tree . . . 8
3.2 An Example Tree: Should We Play Tennis? . . . 8
3.3 Types of Decision Trees . . . 9
3.4 Kinds of Splits . . . 9
3.5 Why Decision Trees? . . . 9

4 Entropy and Information Gain **10**
4.1 Entropy . . . 10
4.2 Information Gain . . . 10
4.3 Worked Example: The Play-Tennis Data . . . 11
4.4 Other Impurity Measures . . . 12

5 Decision Tree Construction **14**
5.1 The Basic Idea: Greedy Recursive Splitting . . . 14
5.2 The ID3 Algorithm . . . 14
5.3 Worked Construction: Play-Tennis . . . 14
5.4 Numeric Attributes: Choosing a Threshold . . . 15
5.5 Stopping Rules . . . 15
5.6 Overfitting and Pruning . . . 16
5.7 Regression Trees . . . 16
5.8 ID3, C4.5 and CART Compared . . . 17
5.9 Beyond One Tree . . . 17
5.10 Code . . . 17

6 Revision Toolkit **18**
6.1 One-Page Summary . . . 18
6.2 Formula Sheet . . . 18
6.3 Commonly Confused Pairs . . . 19
6.4 Short-Answer Practice . . . 19
6.5 Long-Answer Outlines . . . 20
6.6 Multiple-Choice Practice . . . 20
6.7 Glossary . . . 21

1 Introduction to Clustering

In plain words. Walk into a market and you will see vegetables stacked together, fruits together and spices together. Nobody gave you labels, yet you grasp the groups because similar things sit near each other. *Clustering* teaches a computer to find such natural groups in data by itself.

Definition (Clustering). An **unsupervised** learning task that divides unlabelled data into groups (clusters) so that samples in the same cluster are **similar** to each other and **different** from samples in other clusters.

1.1 Clustering vs. Classification

	Clustering	Classification
Learning type	Unsupervised	Supervised
Labels in data	No	Yes
Goal	Discover groups	Predict a known label
Number of groups	Chosen or discovered	Fixed by the labels
Example	Customer segmentation	Spam / not spam

1.2 Applications

- **Customer segmentation:** group shoppers by behaviour for targeted offers.
- **Document and news grouping:** find topics without labelling articles.
- **Image compression and segmentation:** replace pixel colours with a few cluster colours.
- **Anomaly detection:** points far from every cluster are suspicious.
- **Biology and medicine:** group genes or patients with similar profiles.
- **Recommendation:** group users or items with similar taste.

1.3 Similarity Through Distance

Clustering needs a notion of "close". The usual measure is the **Euclidean distance** between points $a = (a_1, \ldots, a_d)$ and $b = (b_1, \ldots, b_d)$:

$$d(a, b) = \sqrt{\sum_{j=1}^{d} (a_j - b_j)^2}.$$

Other choices are Manhattan distance $\sum |a_j - b_j|$ and cosine distance (for text). Because distances drive everything, **features must be scaled** first; otherwise a feature with a large range (income in rupees) overwhelms one with a small range (age in years).

1.4 Main Families of Clustering Methods

Family	Idea	Example algorithm
Partitioning	Split data into k non-overlapping groups around centres	K-Means, K-Medoids
Hierarchical	Build a tree of clusters by merging or splitting step by step	Agglomerative clustering
Density-based	Clusters are dense regions separated by sparse regions; finds odd shapes and noise	DBSCAN
Model-based	Assume data come from a mixture of distributions	Gaussian Mixture Models

This module studies the most popular partitioning method, **K-Means**.

1.5 What Makes a Good Clustering?

- **Cohesion:** points inside a cluster are close together (low within-cluster spread).
- **Separation:** different clusters are far apart.
- **Usefulness:** the groups make sense for the real problem; there is often no single "correct" answer.

2 K-Means Clustering and the Centroid Concept

2.1 The Centroid

Definition (Centroid). The **centroid** of a cluster is the point whose coordinates are the **mean** of the coordinates of all points in the cluster:

$$\mu_k = \frac{1}{|C_k|} \sum_{x \in C_k} x.$$

In plain words. Think of a flat sheet of cardboard with the data points as equal weights stuck on it. The centroid is the *balance point*: the place where you could rest the cardboard on a pin. It is the "centre of the group".

Example: Finding a centroid

Cluster points $(2, 4)$, $(4, 6)$, $(6, 8)$.
$\mu_x = \frac{2+4+6}{3} = 4, \qquad \mu_y = \frac{4+6+8}{3} = 6.$ Centroid $= (4, 6)$.

Facts about centroids:

- The centroid need not be an actual data point. (The *medoid*, used by K-Medoids, *is* an actual data point.)
- The mean is the point that **minimises the sum of squared distances** to the cluster's points. Setting the derivative of $\sum(x_i - c)^2$ to zero gives $c = \bar{x}$. This is why K-Means uses means.
- A centroid moves whenever points join or leave its cluster.

2.2 The K-Means Algorithm

Definition (K-Means). A partitioning algorithm that divides n points into k clusters by repeatedly assigning each point to the nearest centroid and then recomputing each centroid as the mean of its points, until the clusters stop changing.

1. **Choose** k, the number of clusters.
2. **Initialise** k centroids (for example, pick k random data points).
3. **Assignment step:** assign every point to the cluster of its **nearest centroid**.
4. **Update step:** recompute each centroid as the **mean** of the points assigned to it.
5. **Repeat** steps 3 and 4 until assignments no longer change (convergence) or a maximum number of iterations is reached.

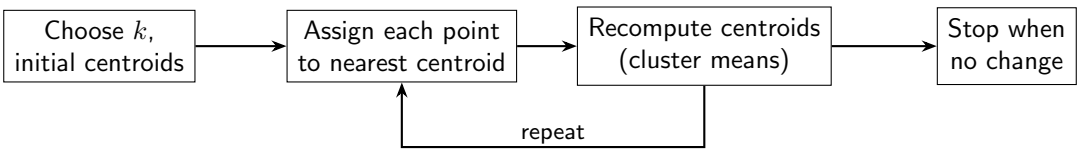


Figure 1: The K-Means loop.

2.3 What K-Means Optimises

K-Means minimises the **within-cluster sum of squares** (WCSS, also called inertia or SSE):

$$J = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2.$$

Each assignment step and each update step can only keep J the same or lower it, so the algorithm always converges, though possibly to a *local* minimum.

2.4 Worked Example

Example: K-Means with $k = 2$ on six points

Points: A$(1,1)$, B$(2,1)$, C$(1,3)$, D$(6,5)$, E$(7,5)$, F$(6,7)$. Start with centroids $c_1 = (1,1)$ (at A) and $c_2 = (1,3)$ (at C).

Iteration 1: assignment (Euclidean distances):

Point	to $c_1 = (1,1)$	to $c_2 = (1,3)$	Cluster
A$(1,1)$	0	2.00	1
B$(2,1)$	1.00	2.24	1
C$(1,3)$	2.00	0	2
D$(6,5)$	6.40	5.39	2
E$(7,5)$	7.21	6.32	2
F$(6,7)$	7.81	6.40	2

Clusters: $\{A, B\}$ and $\{C, D, E, F\}$.

Iteration 1: update. $c_1 = \left(\frac{1+2}{2}, \frac{1+1}{2}\right) = (1.5,\ 1)$; $\quad c_2 = \left(\frac{1+6+7+6}{4}, \frac{3+5+5+7}{4}\right) = (5,\ 5)$.

Iteration 2: assignment.

Point	to $c_1 = (1.5,1)$	to $c_2 = (5,5)$	Cluster
A$(1,1)$	0.50	5.66	1
B$(2,1)$	0.50	5.00	1
C$(1,3)$	2.06	4.47	**1** (moved)
D$(6,5)$	6.02	1.00	2
E$(7,5)$	6.80	2.00	2
F$(6,7)$	7.50	2.24	2

Point C changed cluster. Clusters: $\{A, B, C\}$ and $\{D, E, F\}$.

Iteration 2: update. $c_1 = \left(\frac{1+2+1}{3}, \frac{1+1+3}{3}\right) = (1.33,\ 1.67)$; $\quad c_2 = \left(\frac{6+7+6}{3}, \frac{5+5+7}{3}\right) = (6.33,\ 5.67)$.

Iteration 3: every point stays with the same centroid, so the algorithm has **converged**.

Final WCSS: cluster 1: $0.556 + 0.889 + 1.889 = 3.333$; cluster 2: 3.333; total $J \approx 6.67$. (With one single cluster, $J = 68.17$, so splitting into two removed about 90% of the spread.)

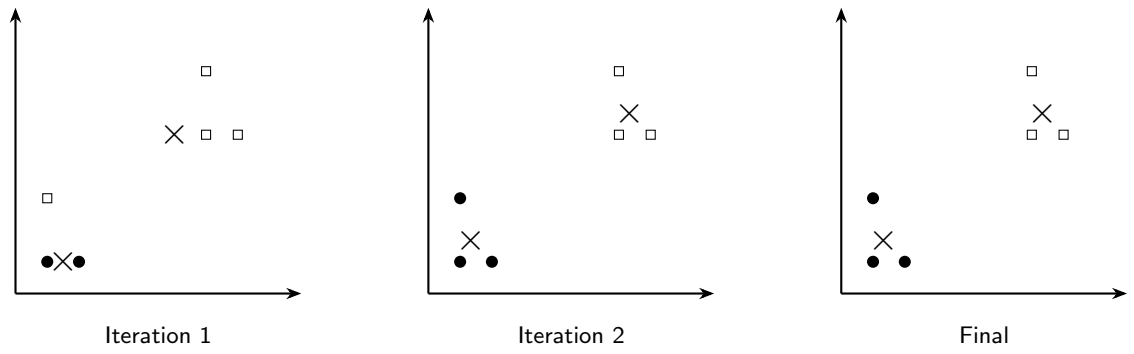


Figure 2: The worked example. Circles and squares are the two clusters; the cross is each cluster's centroid. In the centre panel the point at $(1, 3)$ has just moved to the lower cluster.

2.5 Choosing the Initial Centroids

The result depends on the starting centroids; bad starts can give poor local minima.

- Run the algorithm **several times** with different random starts and keep the run with the lowest J.
- Use **K-Means++**: choose the first centroid at random, then pick each next centroid with probability proportional to its squared distance from the nearest centroid already chosen, so the starting centroids are spread out. It is the default in scikit-learn.

2.6 Choosing k

Elbow method

Run K-Means for $k = 1, 2, 3, \ldots$ and plot WCSS against k. WCSS always falls as k grows (it is 0 when $k = n$), but the drop slows sharply at the natural number of clusters, forming an "elbow".

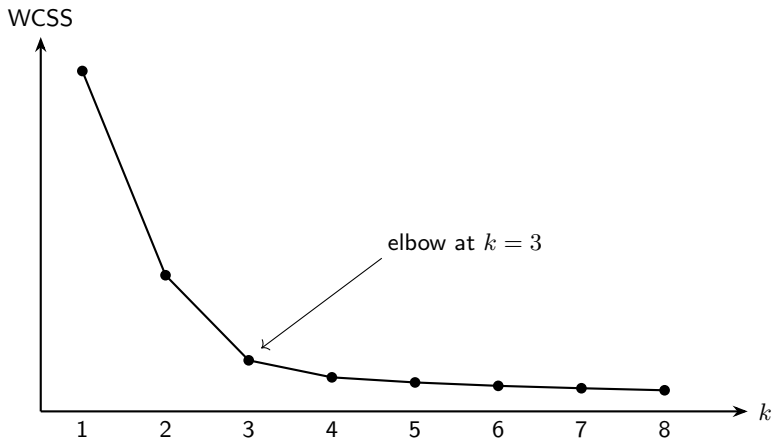


Figure 3: Illustrative elbow plot: the curve bends at $k = 3$.

Silhouette score

For a point i, let $a(i)$ be its average distance to the other points in its own cluster and $b(i)$ its average distance to the points of the nearest other cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \qquad -1 \le s(i) \le 1.$$

Values near $+1$ mean the point is well matched to its cluster; near 0 it is on a boundary; negative means it may be in the wrong cluster. The average s over all points is compared across different

k. Example: $a = 1.2$ and $b = 4.8$ give $s = \frac{4.8-1.2}{4.8} = 0.75$.

Domain knowledge

Often business needs decide k (for instance, three customer tiers).

2.7 Limitations of K-Means

Limitation	Consequence or remedy
Must choose k in advance	Use elbow or silhouette
Sensitive to initial centroids	K-Means++, multiple restarts
Assumes round, similar-sized clusters	Fails on elongated or ring-shaped clusters; use DBSCAN or Gaussian mixtures
Sensitive to outliers (mean is pulled)	Remove outliers or use K-Medoids
Needs numeric data and scaling	Standardise features; encode categories carefully
Hard assignment only	Gaussian mixtures give soft (probabilistic) membership
Struggles in very high dimensions	Reduce dimensions first (PCA)

2.8 Code

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

X = np.array([[1,1],[2,1],[1,3],[6,5],[7,5],[6,7]])
Xs = StandardScaler().fit_transform(X)           # always scale first

km = KMeans(n_clusters=2, init="k-means++", n_init=10, random_state=42).fit(Xs)
print(km.labels_)            # cluster of each point
print(km.cluster_centers_)   # centroids (in scaled space)
print(km.inertia_)           # WCSS

for k in range(2, 6):        # compare different k
    m = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xs)
    print(k, m.inertia_, silhouette_score(Xs, m.labels_))
```

> **Exam tip.** A K-Means numerical asks you to perform one or two iterations. Show the distance table, state the clusters, then compute the new centroids as means, and say when the algorithm stops (no point changes cluster). Mention that the centroid is the mean and need not be a data point.

3 Decision Tree Structure

In plain words. Think of the game "Twenty Questions". You ask a yes/no question, the answer narrows the possibilities, you ask another, and soon you can name the object. A decision tree is that game written down as a flowchart: each question splits the data, and the final answer sits at the end of the path.

Definition (Decision Tree). A supervised learning model that predicts a target by asking a sequence of questions about the input features, arranged as a **tree**. Each internal node tests a feature, each branch is an outcome of the test, and each leaf holds a prediction.

3.1 Anatomy of a Tree

Part	Meaning
Root node	The top node; the first question; it has no parent and contains all the training data
Internal (decision) node	A node that tests a feature and splits the data into children
Branch (edge)	One possible outcome of a test (for example Outlook = Sunny)
Leaf (terminal) node	A node with no children; it gives the final class or value
Parent / child	A node and the nodes created by splitting it
Subtree	A node together with all its descendants
Depth	Number of edges on the longest path from root to a leaf

3.2 An Example Tree: Should We Play Tennis?

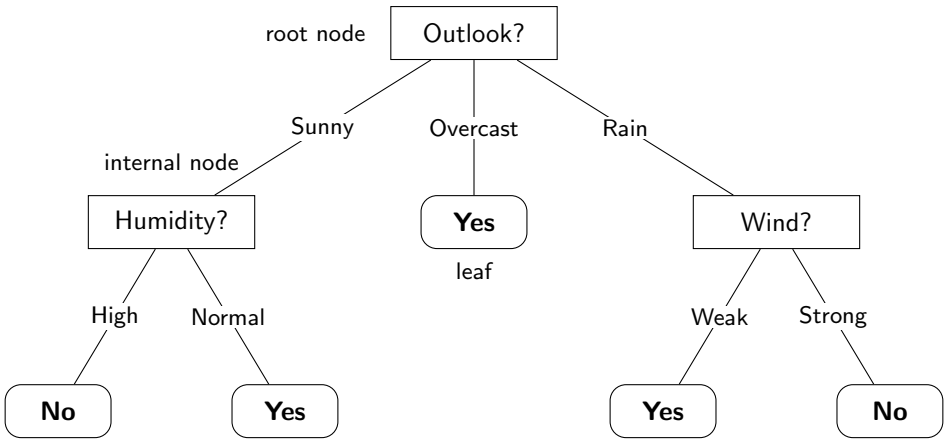


Figure 4: A decision tree of depth 2. Squares are decision nodes; rounded boxes are leaves holding the prediction.

Making a prediction: start at the root and follow the branch that matches the sample until you reach a leaf. For a day with Outlook = Sunny, Humidity = Normal: Outlook → Sunny → Humidity → Normal → **Yes**.

Trees are rules: each root-to-leaf path is an IF–THEN rule, for example `IF Outlook=Overcast THEN Play=Yes`; `IF Outlook=Rain AND Wind=Strong THEN Play=No`. This makes decision trees very easy to explain.

3.3 Types of Decision Trees

- **Classification tree:** target is a category; a leaf predicts the *majority class* of its training samples.
- **Regression tree:** target is a number; a leaf predicts the *mean* of its training samples.

3.4 Kinds of Splits

- **Categorical feature:** one branch per category (ID3, C4.5) or a two-way split such as "Outlook = Sunny vs. others" (CART).
- **Numeric feature:** a threshold test such as Temperature ≤ 54, with two branches.

3.5 Why Decision Trees?

Advantages	Disadvantages
Easy to understand, visualise and explain	Can overfit badly if grown fully
Handle numeric and categorical data	Unstable: a small change in data can change the tree
Little preprocessing: no scaling, tolerant of missing values in some versions	Axis-aligned splits approximate diagonal boundaries poorly
Capture non-linear relationships and interactions	Greedy construction gives no guarantee of the best tree
Perform feature selection automatically	Biased towards features with many values (information gain)
Fast prediction	Single trees are usually less accurate than ensembles

4 Entropy and Information Gain

The key question when building a tree is: *which feature should we test first?* We want the question that makes the groups as **pure** as possible, meaning each group mostly contains one class. Entropy measures impurity, and information gain measures how much a split reduces it.

4.1 Entropy

In plain words. Entropy measures *surprise* or *disorder*. A bag with only red balls has no uncertainty: you always know the colour, so entropy is zero. A bag with half red and half blue is the most unpredictable: entropy is at its maximum.

Definition (Entropy). For a set S in which class i has proportion p_i:

$$H(S) = -\sum_{i=1}^{c} p_i \log_2 p_i.$$

For two classes with proportions p and $1-p$: $H = -p\log_2 p - (1-p)\log_2(1-p)$. By convention $0 \log_2 0 = 0$.

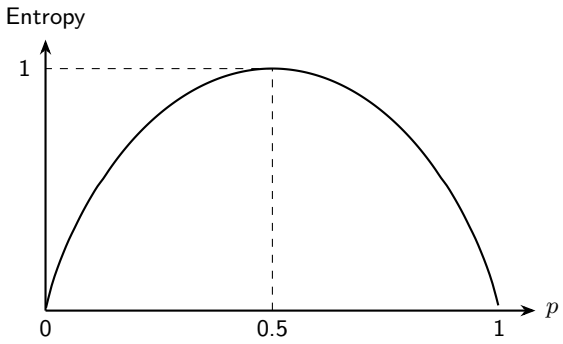


Figure 5: Entropy of a two-class set. It is 0 when the set is pure ($p = 0$ or 1) and 1 when classes are evenly mixed ($p = 0.5$).

Key values (two classes):

p (fraction of one class)	0	0.1	0.5	0.7	0.9
Entropy	0	0.469	1.000	0.881	0.469

Example: a set with 90% positive and 10% negative has $H = -0.9\log_2 0.9 - 0.1\log_2 0.1 = 0.137 + 0.332 = 0.469$. A set with 2 Yes and 2 No has $H = 1$. A pure set (all Yes) has $H = 0$.

4.2 Information Gain

Definition (Information Gain). The reduction in entropy achieved by splitting set S on attribute A:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v),$$

where S_v is the subset with $A = v$. The second term is the **weighted average entropy of the children**.

The attribute with the **highest information gain** is chosen for the split, because it leaves the purest children.

4.3 Worked Example: The Play-Tennis Data

Day	Outlook	Temperature	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Example: Step 1: entropy of the whole data set

9 Yes and 5 No out of 14:

$$H(S) = -\tfrac{9}{14}\log_2\tfrac{9}{14} - \tfrac{5}{14}\log_2\tfrac{5}{14} = 0.410 + 0.531 = 0.940.$$

Example: Step 2: information gain of Outlook

Outlook	Samples	Yes	No	Entropy
Sunny	5	2	3	$-\tfrac{2}{5}\log_2\tfrac{2}{5} - \tfrac{3}{5}\log_2\tfrac{3}{5} = 0.971$
Overcast	4	4	0	0 (pure)
Rain	5	3	2	0.971

Weighted entropy $= \tfrac{5}{14}(0.971) + \tfrac{4}{14}(0) + \tfrac{5}{14}(0.971) = 0.694$.
$\text{Gain}(S, \text{Outlook}) = 0.940 - 0.694 = \mathbf{0.246}$.

Example: Step 3: information gain of the other attributes

Humidity: High has 7 samples (3 Yes, 4 No; entropy 0.985); Normal has 7 samples (6 Yes, 1 No; entropy 0.592). Weighted $= \tfrac{7}{14}(0.985) + \tfrac{7}{14}(0.592) = 0.789$. Gain $= 0.940 - 0.789 =$

0.151.
Wind: Weak has 8 samples (6 Yes, 2 No; entropy 0.811); Strong has 6 samples (3 Yes, 3 No; entropy 1.000). Weighted $= \frac{8}{14}(0.811) + \frac{6}{14}(1.000) = 0.892$. Gain $= 0.940 - 0.892 = \mathbf{0.048}$.
Temperature: Hot has 4 samples (2 Yes, 2 No; entropy 1.000); Mild has 6 samples (4 Yes, 2 No; entropy 0.918); Cool has 4 samples (3 Yes, 1 No; entropy 0.811). Weighted $= \frac{4}{14}(1) + \frac{6}{14}(0.918) + \frac{4}{14}(0.811) = 0.911$. Gain $= 0.940 - 0.911 = \mathbf{0.029}$.

Attribute	Information gain
Outlook	**0.246**
Humidity	0.151
Wind	0.048
Temperature	0.029

Outlook has the highest gain, so it becomes the root of the tree.

4.4 Other Impurity Measures

Gini impurity (used by CART)

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2.$$

It is 0 for a pure set and 0.5 at most for two classes. It is cheaper to compute than entropy (no logarithm) and usually gives very similar trees.

Example: Gini for the play-tennis data
$\text{Gini}(S) = 1 - \left[(\frac{9}{14})^2 + (\frac{5}{14})^2\right] = 1 - (0.413 + 0.128) = 0.459$.
Split on Outlook: Sunny $= 1 - (0.4^2 + 0.6^2) = 0.48$; Overcast $= 0$; Rain $= 0.48$. Weighted $= \frac{5}{14}(0.48) + \frac{4}{14}(0) + \frac{5}{14}(0.48) = 0.343$. Gini reduction $= 0.459 - 0.343 = 0.116$.

Gain ratio (used by C4.5)

Information gain favours attributes with many distinct values (an ID column would split into pure single-sample groups). **Gain ratio** corrects this:

$$\text{SplitInfo}(S, A) = -\sum_{v} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \qquad \text{GainRatio} = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}.$$

For Outlook: SplitInfo $= -2 \cdot \frac{5}{14} \log_2 \frac{5}{14} - \frac{4}{14} \log_2 \frac{4}{14} = 1.577$, so GainRatio $= \frac{0.246}{1.577} = 0.156$.

Measure	Formula	Used by	Notes
Entropy / information gain	$-\sum p \log_2 p$	ID3	Biased to many-valued attributes
Gain ratio	Gain / SplitInfo	C4.5	Corrects that bias
Gini impurity	$1 - \sum p^2$	CART	Fast; binary splits
Variance reduction	Drop in variance of the target	CART regression	For numeric targets

Exam tip. Learn the play-tennis calculation: $H(S) = 0.940$ and the four gains 0.246, 0.151, 0.048, 0.029. Questions usually ask for the entropy and the gain of one

> or two attributes, or to name the root. Always write the gain formula, show the table of subset counts, then the weighted average.

5 Decision Tree Construction

5.1 The Basic Idea: Greedy Recursive Splitting

A tree is built **top-down**. At each node we pick the best question, split the data, and repeat on each child. This is **greedy** (each choice looks only at the current node) and **recursive** (the same procedure is applied to every subset).

5.2 The ID3 Algorithm

1. Start with all training samples at the **root**.
2. **Stopping check:** if all samples in the node have the same class, make it a **leaf** with that class. If no attributes remain (or the node is empty), make it a leaf labelled with the **majority class**.
3. Otherwise compute the **information gain** of every remaining attribute.
4. Choose the attribute with the **highest gain** and make it the node's test.
5. Create one branch per value of that attribute and **partition** the samples among the branches.
6. **Recurse** on each child, using only the remaining attributes.

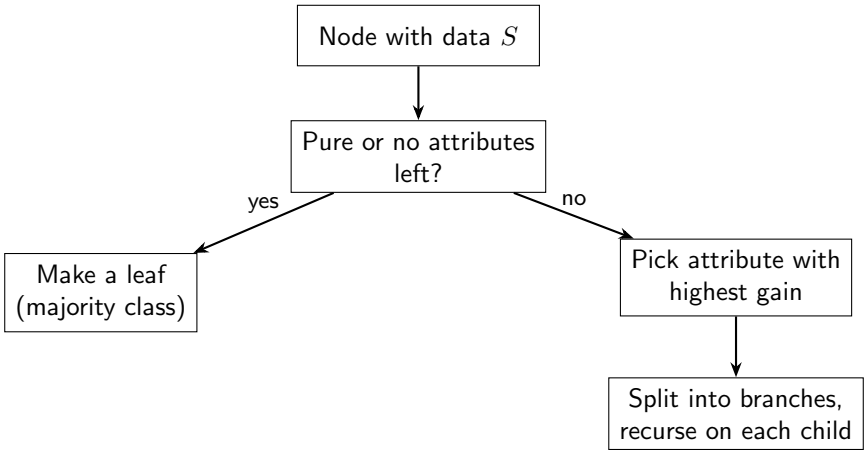


Figure 6: Flow of the tree-building procedure at one node.

5.3 Worked Construction: Play-Tennis

Root. From Chapter 4, Outlook has the highest gain (0.246), so it becomes the root with three branches: Sunny, Overcast, Rain.

> **Example: Branch Overcast**
> All four samples (D3, D7, D12, D13) are Yes. The node is pure, so it becomes a leaf: **Yes**.

> **Example: Branch Sunny: samples D1, D2, D8, D9, D11 (2 Yes, 3 No; $H = 0.971$)**

Attribute	Gain	Reason
Humidity	**0.971**	High: D1, D2, D8 all No; Normal: D9, D11 all Yes; both children pure
Temperature	0.571	Hot: both No ($H = 0$); Mild: 1 Yes, 1 No ($H = 1$); Cool: 1 Yes ($H = 0$); weighted 0.400
Wind	0.020	Weak: 1 Yes, 2 No (0.918); Strong: 1 Yes, 1 No (1); weighted 0.951

Humidity is chosen. Its children are pure: High → **No**, Normal → **Yes**.

Example: Branch Rain: samples D4, D5, D6, D10, D14 (3 Yes, 2 No; $H = 0.971$)

Attribute	Gain	Reason
Wind	**0.971**	Weak: D4, D5, D10 all Yes; Strong: D6, D14 all No; both children pure
Humidity	0.020	High: 1 Yes, 1 No (1); Normal: 2 Yes, 1 No (0.918); weighted 0.951
Temperature	0.020	Mild: 2 Yes, 1 No (0.918); Cool: 1 Yes, 1 No (1); weighted 0.951

Wind is chosen. Weak → **Yes**, Strong → **No**.

Result: the tree drawn in Chapter 3. Every leaf is pure, so the construction stops. Notice that **Temperature never appears**: the tree has selected features automatically. Reading it as rules:

- IF Outlook = Overcast THEN Play = Yes.
- IF Outlook = Sunny AND Humidity = High THEN Play = No.
- IF Outlook = Sunny AND Humidity = Normal THEN Play = Yes.
- IF Outlook = Rain AND Wind = Weak THEN Play = Yes.
- IF Outlook = Rain AND Wind = Strong THEN Play = No.

5.4 Numeric Attributes: Choosing a Threshold

For a numeric feature, sort the values and test a threshold **midway between neighbouring values whose class changes**.

Example: Splitting on temperature

Values and classes: 40 (No), 48 (No), 60 (Yes), 72 (Yes), 80 (Yes), 90 (No). The set has 3 Yes and 3 No, so $H(S) = 1$.

Class changes between 48 and 60, and between 80 and 90, so the candidate thresholds are 54 and 85.

Threshold 54: left $\{40, 48\}$ is pure ($H = 0$); right $\{60, 72, 80, 90\}$ has 3 Yes, 1 No ($H = 0.811$). Weighted $= \frac{2}{6}(0) + \frac{4}{6}(0.811) = 0.541$. Gain $= 1 - 0.541 = \mathbf{0.459}$.

Threshold 85: left $\{40, 48, 60, 72, 80\}$ has 2 No, 3 Yes ($H = 0.971$); right $\{90\}$ is pure. Weighted $= \frac{5}{6}(0.971) = 0.809$. Gain $= 1 - 0.809 = 0.191$.

The best split is **Temperature** ≤ 54.

5.5 Stopping Rules

The tree stops growing a branch when:

- the node is **pure** (one class);
- no features are left to test;
- the node has **too few samples** (a minimum, for instance 5);
- the **maximum depth** has been reached;
- the best split gives **too little gain** (below a minimum threshold).

5.6 Overfitting and Pruning

A tree grown until every leaf is pure can memorise noise, one leaf per odd sample, and generalise badly.

Example: Full tree vs. pruned tree (illustrative)

Tree	Training accuracy	Test accuracy
Fully grown (depth 12)	100%	78%
Pruned (depth 4)	90%	85%

The pruned tree is simpler, loses some training accuracy, and **generalises better**.

Pruning type	How it works
Pre-pruning (early stopping)	Stop growing early with limits such as `max_depth`, `min_samples_leaf`, minimum gain
Post-pruning	Grow the full tree, then remove branches that do not improve validation performance (for example cost-complexity pruning in CART)

Choose the limits with cross-validation, never with the test set.

5.7 Regression Trees

For a numeric target, a split is judged by **variance reduction**, and each leaf predicts the **mean** of its samples.

Example: Variance reduction

Node values: $10, 12, 20, 22$. Mean $= 16$; squared deviations $36, 16, 16, 36$; variance $= \frac{104}{4} = 26$.

Split into $\{10, 12\}$ and $\{20, 22\}$: means 11 and 21; each child has variance 1. Weighted variance $= 1$. Reduction $= 26 - 1 = 25$. Leaves predict 11 and 21.

5.8 ID3, C4.5 and CART Compared

	ID3	C4.5	CART
Split criterion	Information gain	Gain ratio	Gini (classification), variance (regression)
Branching	Multiway	Multiway; binary for numeric	Always binary
Numeric attributes	No (must discretise)	Yes, thresholds	Yes, thresholds
Missing values	Not handled	Handled	Surrogate splits / handled by library
Pruning	None	Post-pruning	Cost-complexity pruning
Regression	No	No	Yes

5.9 Beyond One Tree

A single tree has high variance. **Ensembles** fix this: a **random forest** trains many trees on random subsets of data and features and averages their votes (reducing variance); **gradient boosting** adds small trees one by one, each correcting the previous errors (reducing bias).

5.10 Code

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import GridSearchCV
import pandas as pd

X = pd.get_dummies(df[["Outlook", "Temperature", "Humidity", "Wind"]])
y = df["Play"]

tree = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=0)
tree.fit(X, y)

print(export_text(tree, feature_names=list(X.columns)))   # the rules
print(dict(zip(X.columns, tree.feature_importances_)))     # feature importance

grid = GridSearchCV(DecisionTreeClassifier(random_state=0),
                    {"max_depth": [2, 3, 4, 5, None],
                     "min_samples_leaf": [1, 5, 10],
                     "criterion": ["gini", "entropy"]}, cv=5)
grid.fit(X_train, y_train)               # prune by cross-validation
print(grid.best_params_)
```

Watch out. Scikit-learn's trees use binary splits and need numeric input, so categorical features must be one-hot encoded; this produces tests such as "Outlook_Sunny ≤ 0.5" rather than three-way splits. The textbook ID3 tree and the library tree can therefore look different even when they agree on predictions.

Exam tip. The usual long question is "Construct a decision tree for the given data using ID3". Steps: (1) entropy of the whole set; (2) gain of every attribute; (3) pick the highest as root; (4) repeat for each impure branch; (5) draw the tree and write the rules. Show the subset-count table for each attribute; examiners award marks for the working.

6 Revision Toolkit

6.1 One-Page Summary

Topic	Remember this
Clustering	Unsupervised grouping of similar unlabelled samples; needs a distance measure and scaled features.
Centroid	Mean of the points of a cluster; the balance point; need not be a data point.
K-Means	Choose k; initialise centroids; assign to nearest centroid; recompute means; repeat until no change. Minimises WCSS $= \sum \|x - \mu_k\|^2$.
Choosing k	Elbow method (bend in WCSS curve), silhouette score $\frac{b-a}{\max(a,b)}$, domain knowledge.
K-Means limits	Fixed k, initial centroids matter (K-Means++), spherical clusters, outliers.
Tree structure	Root, internal nodes (tests), branches (outcomes), leaves (predictions); path = IF–THEN rule.
Entropy	$H = -\sum p_i \log_2 p_i$; 0 for pure, 1 for 50/50 (two classes).
Information gain	$H(S) - \sum \frac{\|S_v\|}{\|S\|} H(S_v)$; choose the highest.
Construction	ID3: greedy, recursive; stop when pure, no attributes, or limits reached; prune to avoid overfitting.
Play-tennis	$H(S) = 0.940$; gains: Outlook 0.246, Humidity 0.151, Wind 0.048, Temperature 0.029.

6.2 Formula Sheet

Name	Formula
Euclidean distance	$d(a,b) = \sqrt{\sum_j (a_j - b_j)^2}$
Centroid	$\mu_k = \dfrac{1}{\|C_k\|} \sum_{x \in C_k} x$
WCSS (inertia)	$J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2$
Silhouette	$s = \dfrac{b-a}{\max(a,b)}$
Entropy	$H(S) = -\sum p_i \log_2 p_i$
Information gain	$\text{Gain} = H(S) - \sum \frac{\|S_v\|}{\|S\|} H(S_v)$
Gini impurity	$1 - \sum p_i^2$
Split information	$-\sum \frac{\|S_v\|}{\|S\|} \log_2 \frac{\|S_v\|}{\|S\|}$
Gain ratio	Gain/SplitInfo
Variance reduction	$\text{Var}(S) - \sum \frac{\|S_v\|}{\|S\|} \text{Var}(S_v)$

6.3 Commonly Confused Pairs

Pair	First	Second
K-Means vs KNN	Unsupervised clustering	Supervised classifier
Centroid vs medoid	Mean; may not be a data point	Actual data point
Clustering vs classification	No labels; discovers groups	Labels known; predicts class
Entropy vs information gain	Impurity of a set	Reduction in impurity after a split
Entropy vs Gini	Uses logarithms; max 1 for two classes	No logarithm; max 0.5 for two classes
Root vs leaf	First test; holds all data	Final prediction; no children
Pre-pruning vs post-pruning	Stop growth early	Grow fully, then cut back
Classification vs regression tree	Leaf is majority class	Leaf is mean value

6.4 Short-Answer Practice

1. **What is clustering?** Unsupervised grouping of data so that similar samples fall in the same group.
2. **What is a centroid?** The mean position of all points in a cluster.
3. **State the K-Means steps.** Initialise k centroids; assign points to nearest centroid; recompute centroids; repeat until no change.
4. **What does K-Means minimise?** The within-cluster sum of squared distances to the centroids.
5. **How is k chosen?** Elbow method, silhouette score, or domain knowledge.
6. **Name two limitations of K-Means.** Needs k in advance; sensitive to initial centroids and outliers; assumes round clusters.
7. **What is a decision tree?** A flowchart-like model that predicts through a sequence of feature tests ending in leaves.
8. **Define root, internal and leaf nodes.** Root: first test; internal: further tests; leaf: final prediction.
9. **What is entropy?** A measure of impurity or uncertainty of a set of class labels.
10. **What is the entropy of a pure node?** Zero.
11. **Define information gain.** Entropy before a split minus the weighted entropy of the children.
12. **Which attribute does ID3 choose?** The one with the highest information gain.
13. **Why prune a tree?** To reduce overfitting and improve test accuracy.
14. **Give two advantages of decision trees.** Easy to interpret; handle numeric and categorical data without scaling.

6.5 Long-Answer Outlines

Question	Skeleton that earns full marks
Explain the K-Means algorithm with an example	Definition; centroid; steps and flowchart; worked iterations with distance table; WCSS; choosing k (elbow); limitations.
Explain the centroid concept	Definition as mean; small numerical; balance-point idea; need not be a data point; link to minimising squared distance.
Explain decision tree structure	Definition; root, internal nodes, branches, leaves; example tree; prediction by traversal; rules; advantages and disadvantages.
Explain entropy and information gain with an example	Formulas; entropy curve; play-tennis numbers; gains of all attributes; conclusion on the root.
Explain how a decision tree is constructed	ID3 steps; stopping rules; worked branches; numeric thresholds; overfitting and pruning; ID3 vs C4.5 vs CART.

6.6 Multiple-Choice Practice

1. K-Means is a(n): (a) supervised algorithm (b) unsupervised algorithm (c) reinforcement algorithm (d) regression algorithm
2. A centroid is: (a) the farthest point (b) a random point (c) the mean of a cluster's points (d) the median label
3. K-Means minimises: (a) within-cluster sum of squares (b) number of features (c) entropy (d) accuracy
4. The centroid of $(2,4),(4,6),(6,8)$ is: (a) $(3,5)$ (b) $(4,6)$ (c) $(6,8)$ (d) $(12,18)$
5. The elbow method is used to choose: (a) the number of clusters (b) the learning rate (c) tree depth (d) the distance metric
6. K-Means needs which input in advance? (a) labels (b) a tree depth (c) the number of clusters k (d) a threshold
7. The silhouette score lies between: (a) 0 and 1 (b) -1 and 1 (c) 0 and 100 (d) $-\infty$ and ∞
8. The topmost node of a decision tree is the: (a) root (b) leaf (c) branch (d) subtree
9. A leaf node contains: (a) a test (b) a prediction (c) a threshold (d) a distance
10. The entropy of a pure node is: (a) 0 (b) 0.5 (c) 1 (d) ∞
11. The maximum entropy of a two-class set is: (a) 0.5 (b) 1 (c) 2 (d) 9
12. Information gain equals: (a) parent entropy − weighted child entropy (b) child entropy − parent entropy (c) Gini − entropy (d) the number of leaves
13. ID3 selects the attribute with the: (a) lowest gain (b) lowest entropy gain ratio (c) highest information gain (d) most values
14. A set with 9 Yes and 5 No has entropy about: (a) 0.5 (b) 0.940 (c) 1.0 (d) 0.246
15. Gini impurity is used in: (a) ID3 (b) CART (c) K-Means (d) Apriori
16. Pruning is used to: (a) reduce overfitting (b) increase depth (c) add features (d) scale data
17. In the play-tennis data the root attribute is: (a) Wind (b) Humidity (c) Outlook (d) Temperature
18. A node with 2 Yes and 2 No has entropy: (a) 0 (b) 0.5 (c) 0.811 (d) 1

> **Note. Answers:** 1-b, 2-c, 3-a, 4-b, 5-a, 6-c, 7-b, 8-a, 9-b, 10-a, 11-b, 12-a, 13-c, 14-b, 15-b, 16-a, 17-c, 18-d.

6.7 Glossary

Term	Meaning
Branch	Outcome of a test connecting a node to a child
Centroid	Mean position of a cluster's points
Cluster	Group of similar samples
Convergence	State where K-Means assignments stop changing
Elbow method	Choosing k at the bend of the WCSS curve
Entropy	Measure of impurity of a set
Gain ratio	Information gain divided by split information
Gini impurity	$1 - \sum p^2$, a CART impurity measure
Greedy algorithm	Chooses the best option at each step without looking ahead
Inertia / WCSS	Sum of squared distances of points to their centroids
Information gain	Reduction in entropy from a split
Internal node	Node that tests a feature
K-Means++	Initialisation spreading starting centroids apart
Leaf	Terminal node with a prediction
Medoid	Cluster centre that is an actual data point
Pruning	Cutting a tree back to reduce overfitting
Purity	Degree to which a node contains one class
Root	Top node of the tree
Silhouette score	Measure of how well points fit their cluster
Split	Dividing a node's data by a feature test

End of Module 4 — Clustering and Decision Trees
Introduction to Machine Learning • Prof. Rajesh Nag